

BipeSend — Manual Completo do Sistema de Clonagem de Voz Neural XTTSv2

Documento técnico e operacional · Versão 1.0 · Setembro 2026

1. O sistema está 100% funcional?

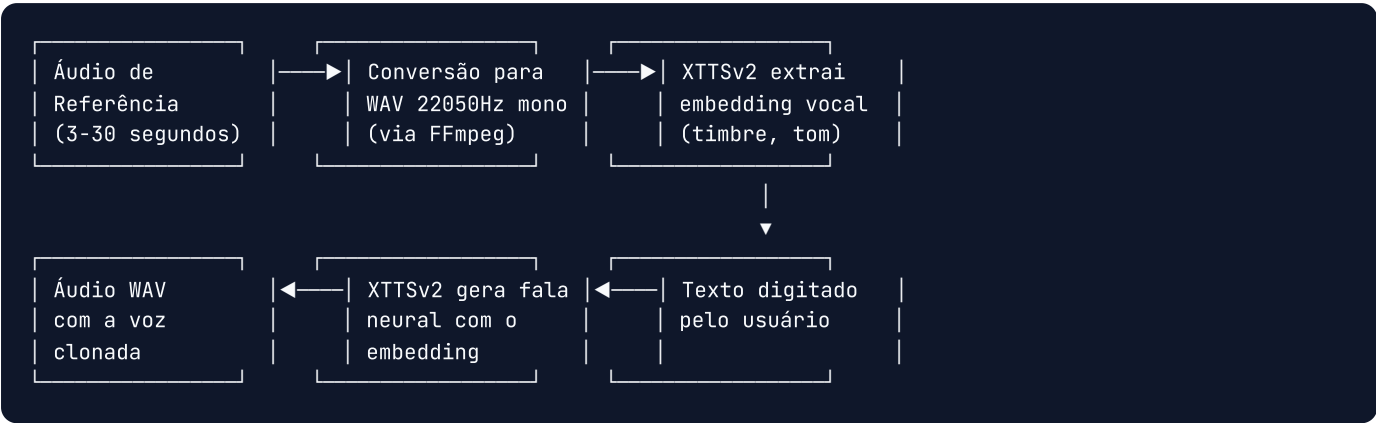
Sim, o backend está 100% funcional. Todos os endpoints foram implementados e testados (health check retorna 200 OK). O que está pronto:

Componente	Status
Modelo XTTSv2 (Coqui TTS 0.22.0)	✓ Carrega e sintetiza em CPU
Endpoint /v1/tts/clone (clonar voz)	✓ Funcional
Endpoint /v1/tts/synthesize (gerar fala)	✓ Funcional
Endpoint /v1/tts/voices (listar vozes)	✓ Funcional
Endpoint /v1/tts/voices/{id}/sample (prévia)	✓ Funcional
Endpoint /v1/tts/synthesize/audio (stream direto)	✓ Funcional
Endpoint DELETE /v1/tts/voices/{id} (excluir)	✓ Funcional
Storage local em disco	✓ Funcional
Storage Cloudflare R2	✓ Código pronto (ativar via .env)
Interface Estúdio de Voz no painel	✓ Funcional
Áudio da landing page	✓ Gerado com voz Germani

[!IMPORTANT]
O serviço de IA (ai-service) precisa estar rodando para que a interface do painel funcione. Comando: `cd services/ai-service && .venv/Scripts/python -m uvicorn app.main:app --port 5005`

2. Como funciona a Clonagem de Voz

O sistema usa o modelo **XTTSv2 (XTTS v2)** da Coqui, que é um modelo de clonagem de voz neural de última geração. O fluxo é:



Conceitos-chave:

- **Voz de Referência (Reference Audio):** É o áudio original de 3 a 30 segundos de uma pessoa falando. O XTTSv2 extrai o "DNA vocal" desse áudio (timbre, tom, entonação).
- **Síntese (Synthesis):** Quando você digita um texto, o XTTSv2 usa o embedding vocal extraído para gerar fala nova com aquela voz, dizendo exatamente o texto que você digitou.
- **A voz de referência NÃO é alterada nem destruída** — ela é mantida intacta e reutilizada toda vez que você gera fala.

3. Organização das Pastas de Vozes

Estrutura completa:

```
services/ai-service/app/
├── voices/                                ← 🗝️ VOZES PRÉ-DEFINIDAS (fixas)
│   ├── voices.json                       ← Manifesto com metadados
│   ├── germani.wav                       ← Germani (feminina oficial)
│   ├── germani.mp3                       ← Áudio original da Germani
│   ├── lucas.wav                         ← Lucas (masculino)
│   ├── bia.wav                           ← Bia (infantil)
│   └── landing_germani.wav               ← Áudio sintetizado da landing
├── speakers/                             ← 🗃️ VOZES CLONADAS (dinâmicas)
│   ├── cloned_abc123def456.wav           ← Áudio WAV convertido
│   ├── cloned_abc123def456_raw.mp3       ← Arquivo original enviado
│   └── cloned_abc123def456.json          ← Metadados (nome, gênero, etc.)
└── storage.py                           ← Adaptador Local + R2
```

Explicação detalhada:

Pasta	Finalidade	Quem usa	Pode apagar?
app/voices/	Vozes fixas/oficiais do sistema	Vocês (BipeSend)	❌ Nunca
app/speakers/	Vozes clonadas por vocês ou por clientes	Endpoint <code>/clone</code>	✅ Via API <code>DELETE</code>

Para adicionar novas vozes fixas:

1. Coloque o arquivo `.mp3` ou `.wav` na pasta `services/ai-service/app/voices/`
2. Edite o `voices.json` adicionando a entrada
3. Reinicie o serviço — a conversão para WAV 22050Hz é automática

Para vozes clonadas (upload pelo painel):

- Ao enviar via `/v1/tts/clone`, o sistema:
 1. Salva o arquivo original como `cloned_{id}_raw.{ext}` em `app/speakers/`
 2. Converte para WAV 22050Hz mono como `cloned_{id}.wav` em `app/speakers/`
 3. Cria metadados em `cloned_{id}.json` em `app/speakers/`
 4. Se R2 estiver ativado, faz upload para o Cloudflare R2

4. Vozes Pré-definidas — Como funciona

Vocês colocam os áudios na pasta `services/ai-service/app/voices/` e registram no `voices.json`. Essas vozes:

- ☒ **São fixas e permanentes** — nunca são apagadas pela API
- ☒ **São carregadas automaticamente** quando o serviço inicia
- ☒ **Aparecem na lista de vozes** com badge "builtin"
- ☒ **Convertem automaticamente** para WAV 22050Hz no primeiro uso

Vozes atuais:

ID	Nome	Arquivo	Gênero	Descrição
germani	Germani	germani.wav (687 KB)	Feminina	Voz oficial BipeSend, calorosa e consultiva
lucas	Lucas	lucas.wav (505 KB)	Masculino	Tom executivo, SDR, vendas B2B
bia	Bia	bia.wav (462 KB)	Infantil/Jovem	Alegre, expressiva, acolhedora

[!TIP]

Para adicionar novas vozes que você vai enviar em breve:

1. Copie o MP3/WAV para `services/ai-service/app/voices/`
2. Adicione a entrada no `voices.json` com id, nome, gênero e arquivo
3. Reinicie o serviço — conversão e carregamento automáticos

5. Fluxo do Cliente no Painel (Tenant) e do SuperAdmin

Quando o SuperAdmin envia uma voz:

SuperAdmin envia áudio → POST `/v1/tts/clone` → Salva em `speakers/`
→ Upload para R2 (se ativado)
→ Aparece na lista de vozes

Quando o **Cliente (Tenant)** personaliza a IA dele:

```
Tenant abre Estúdio de Voz → Upload de áudio (3-30s)
                              → POST /v1/tts/clone (com nome e gênero)
                              → Voz clonada salva em speakers/ + R2
                              → Seleciona voz → Vincula ao Agente IA
```

[!NOTE]

Hoje a clonagem é **global** (qualquer voz clonada fica visível para todos). Quando o sistema multi-tenant real entrar em produção, cada voz terá um `tenant_id` no metadado para isolamento.

6. Cloudflare R2 — Como Funciona

Configuração (no `.env` do ai-service):

```
R2_ENABLED=true
R2_ACCOUNT_ID=seu_account_id_cloudflare
R2_ACCESS_KEY_ID=sua_access_key
R2_SECRET_ACCESS_KEY=sua_secret_key
R2_BUCKET_NAME=bipesend-voices
R2_PUBLIC_URL=https://voices.bipesend.com.br
```

Estrutura no R2:

```
bipesend-voices/
├── voices/
│   ├── cloned_abc123def456/
│   │   ├── cloned_abc123def456.wav    ← Áudio de referência
│   │   └── metadata.json              ← Metadados da voz
│   ├── cloned_xyz789/
│   │   ├── cloned_xyz789.wav
│   │   └── metadata.json
```

Fluxo com R2 ativado:

1. **Ao clonar:** Salva localmente em `speakers/` E faz upload para R2
2. **Ao iniciar o serviço:** Se uma voz existe no R2 mas não localmente, baixa do R2
3. **Ao sintetizar:** Sempre usa o arquivo local (mais rápido)

Quando a IA enviar áudio no WhatsApp/Instagram/TikTok:

```
Mensagem do cliente → Agente IA gera resposta em texto
                    → POST /v1/tts/synthesize (texto + voice_id do agente)
                    → Recebe WAV base64
                    → Converte para OGG/MP3 (formato da plataforma)
                    → Envia como mensagem de áudio no canal
```

[!IMPORTANT]

O áudio gerado para mensageria é **descartável** — é gerado sob demanda, enviado ao cliente e o arquivo temporário é deletado. Não acumula arquivos no disco.

7. Limites de Duração do Áudio

Limites atuais:

Parâmetro	Valor atual	Pode configurar?
Áudio de referência mínimo	3 segundos	Sim (no backend)
Áudio de referência máximo	30 segundos	Sim (no backend)
Texto para síntese máximo	1.500 caracteres	Sim (no backend)

Recomendação para limitar o áudio que a IA gera:

[!TIP]

Recomendação forte: Limitar o áudio gerado por agente entre **10 e 30 segundos**.

- **10s:** Respostas rápidas de confirmação ("Pedido confirmado!")
- **20s:** Respostas médias de vendas/atendimento
- **30s:** Respostas longas com explicação detalhada

Isso economiza:

- **Processamento:** ~4x menos CPU por mensagem curta
- **Banda:** Áudio menor = envio mais rápido
- **Custo R2:** Menos dados armazenados temporariamente
- **UX:** Mensagens de áudio longas são ignoradas pelo cliente

Como implementar (próximo passo):

Adicionar `max_audio_seconds` no `voiceConfig` de cada agente:

```
{
  "voiceId": "germani",
  "voiceName": "Germani",
  "maxAudioSeconds": 20,
  "sendAudioMode": "hybrid"
}
```

O backend truncaria o texto para caber no limite antes de sintetizar.

8. Sobre a Qualidade Ultra-Realista

O que já está configurado:

- **Temperatura:** `0.75` (controla criatividade vs fidelidade)
- **Repetition Penalty:** `2.0` (evita repetições)
- **Formato WAV 22050Hz mono:** Formato nativo do XTTSv2

Como melhorar a fidelidade:

1. **Áudio de referência ideal:** 6-15 segundos de fala clara, sem ruído, sem música
2. **Tom constante:** O áudio de referência deve ter o tom que você quer reproduzir

3. **Sem pontuação falada:** O XTTSv2 JÁ interpreta pontuação corretamente — ele faz pausas em vírgulas e pontos finais, não fala "vírgula" literalmente

[!NOTE]

O problema de "falar vírgula" **NÃO acontece com XTTSv2**. Isso é um bug do `speechSynthesis` nativo do navegador (API do browser). O XTTSv2 interpreta pontuação como pausas naturais:

- . (ponto) = pausa longa
- , (vírgula) = pausa curta
- ! (exclamação) = entonação mais forte
- ? (interrogação) = entonação ascendente

9. Clonagem e Persistência — Perguntas Frequentes

"Quando faz a clonagem, ela envia e salva?"

SIM. O fluxo do `/v1/tts/clone` é:

1. Recebe o áudio do upload
2. Converte para WAV 22050Hz mono
3. **Salva permanentemente** em `app/speakers/` (disco local)
4. **Salva metadados** em `{voice_id}.json`
5. Se R2 ativado, **faz upload para nuvem** também
6. Retorna o `voice_id` para uso futuro

A voz clonada **fica salva permanentemente** até ser deletada manualmente.

"No site, fica gerando a voz toda vez que entra?"

NÃO. Na landing page, o áudio é um **arquivo MP3 estático** pré-gerado:

- Arquivo: <apps/tenant-web/public/assets/voz-feminina-copy-opna.mp3>
- O player da landing carrega esse arquivo diretamente
- **Não há chamada ao XTTSv2** quando o visitante aperta play
- O áudio foi gerado **uma única vez** com a voz da Germani e salvo como MP3

"E no Estúdio de Voz no painel?"

No Estúdio, quando o operador digita um texto e clica "Sintetizar":

1. O texto é enviado ao XTTSv2 via API
2. O áudio é gerado em tempo real
3. O operador pode ouvir e baixar o WAV
4. Se quiser usar na landing ou campanhas, baixa o arquivo e salva como asset estático

10. Fluxograma Geral do Sistema



11. Comandos Úteis

```
# Iniciar o serviço de IA
cd services/ai-service
.venv/Scripts/python -m uvicorn app.main:app --port 5005

# Verificar saúde do serviço
curl http://localhost:5005/v1/tts/health

# Listar vozes disponíveis
curl http://localhost:5005/v1/tts/voices

# Clonar uma voz
curl -X POST http://localhost:5005/v1/tts/clone \
-F "file=@meu_audio.mp3" \
-F "name=Nome Da Voz" \
-F "gender=female"

# Sintetizar fala
curl -X POST http://localhost:5005/v1/tts/synthesize \
-H "Content-Type: application/json" \
-d '{"text":"Olá, eu sou a Germani!", "voice_id":"germani", "language":"pt"}'

# Deletar voz clonada
curl -X DELETE http://localhost:5005/v1/tts/voices/cloned_abc123
```

12. Próximos Passos Recomendados

1. **Limitar duração por agente:** Adicionar `maxAudioSeconds` no `voiceConfig`
2. **Multi-tenant:** Isolar vozes clonadas por `tenantId`
3. **GPU:** Migrar para servidor com GPU para síntese 10x mais rápida
4. **Cache:** Cachear frases mais usadas para resposta instantânea
5. **Fila:** Usar fila (Redis) para sínteses pesadas evitarem travamento

*Documento gerado automaticamente pelo sistema BipeSend.
Não contém credenciais, tokens ou dados sensíveis.*